MOVER Project: Weekly Update

DiscoNet: Reproducing and Understanding the Authors' Pipeline

Li et al., NeurIPS 2021

LUIS ALBERTO VÁSQUEZ VARGAS,
Wellington Lobato, Guilherme Maia,
Allan M. de Souza, Edmundo Madeira

November 4, 2026 — Week 45

Agenda

01 **Current Steps**
Task, objectives, and progress

02 **LiDAR Preprocessing**
Point clouds to occupancy grids

03 **Reusable Blocks**
Units, helpers, and stage composition

04 **Encoder**
Multiscale feature extraction

05 **DiscoGraph**
Alignment and cooperative feature fusion

06 **Decoder**
Upsampling and skip connections

07 **Experiments**
Metrics, dataset analysis, and results

08 **Next Steps**
Actions, deliverables, and validation

Appendix Conv2D vs Conv3D

Current Steps

Current Steps

Current task

Understand the DiscoNet pipeline and run inference to reproduce the authors' results and develop ideas for extensions.

Objective

Identify the ideas and design decisions behind the authors' architecture and reuse their proposed dataset to explore extensions.

Method or approach

Study the pipeline top-down: identify its main components, represent them as function calls, and recursively decompose each into smaller reusable functions.

Current Steps

Progress so far

Organized the pipeline into reusable functions and ran inference on V2X-Sim 2.0.

Partial results or evidence

Evaluated 1,000 test frames; results closely match the maintainer's checkpoint evaluation.

Current difficulties

Finding useful extensions.

Support needed

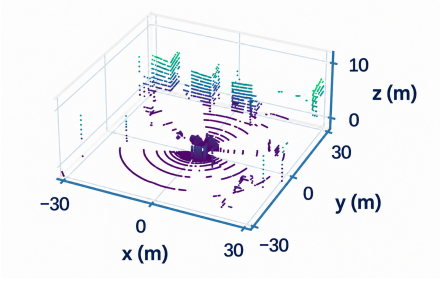
Guidance on extension ideas.

LiDAR Preprocessing

LiDAR Preprocessing

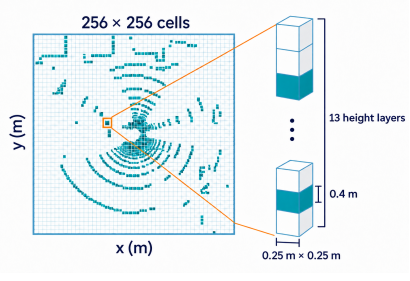
Preprocessing converts point clouds stored in PCD format into fixed-size occupancy grids for DiscoNet.

LiDAR point cloud



One LiDAR scan per vehicle, preprocessed independently.

Fixed-size occupancy grid



Each cell stores occupancy in 13 height layers.
Teal: occupied (1).
White: not occupied (0).

Source: AI generated illustrations

Voxelization

`voxelize_occupy`: `PointCloud` $\longrightarrow$ `OccupancyGrid3D`

Units: meters

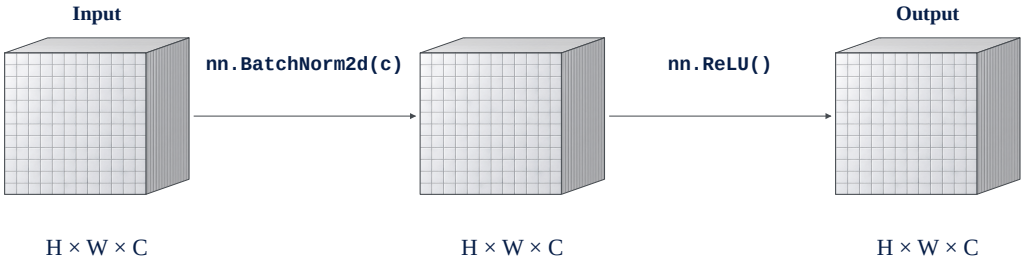
1. **Validate Coordinates**
   Validate 3D coordinates; reject missing or infinite values.
2. **Remove points near the origin**
   In the sensor frame, remove points where $-1 < x < 1$ **AND** $-1 < y < 1$.
3. **Crop the scan**
   Keep points satisfying $-32 < x < 32$ **AND** $-32 < y < 32$ **AND** $-3 < z < 2$.
4. **Start with zeros**
   Create a grid of 256×256 cells with 13 height layers, all starting at 0.
5. **Fill the grid**
   For each retained point, find its $0.25 \times 0.25 \times 0.4$ voxel and set `grid[i,j,k] = 1`.
   Cells without points stay at 0.

Source: CoPerception, `voxelize_occupy` — github.com/coperception/coperception

Reusable Blocks

Normalization and Activation

`norm_relu(c)` c = number of channels




```
def norm_relu(c):
    return nn.Sequential(nn.BatchNorm2d(c), nn.ReLU())
```

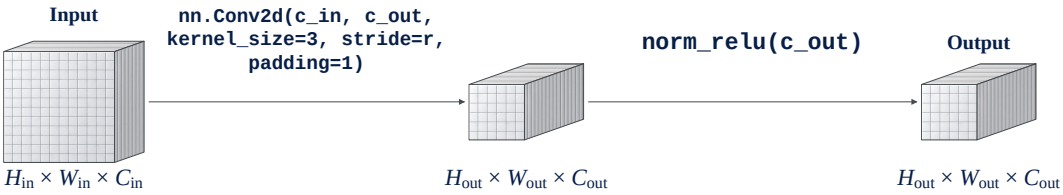
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

2D Unit

`unit_2d(c_in, r, c_out)`




```python
def unit_2d(c_in, r, c_out):
    conv = nn.Conv2d(c_in, c_out, kernel_size=3, stride=r, padding=1)
    return nn.Sequential(conv, *norm_relu(c_out))
```

Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

2D Unit: Variants

Each variant reuses `unit_2d`.

Initial: Set Channels

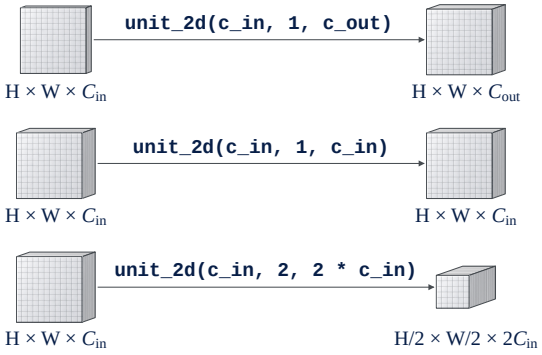
```
def unit_2d_initial(c_in, c_out):
    return unit_2d(c_in, 1, c_out)
```

Keep: Preserve Shape

```
def unit_2d_keep(c_in):
    return unit_2d(c_in, 1, c_in)
```

Down: Halve H and W, Double C

```
def unit_2d_down(c_in):
    return unit_2d(c_in, 2, 2 * c_in)
```



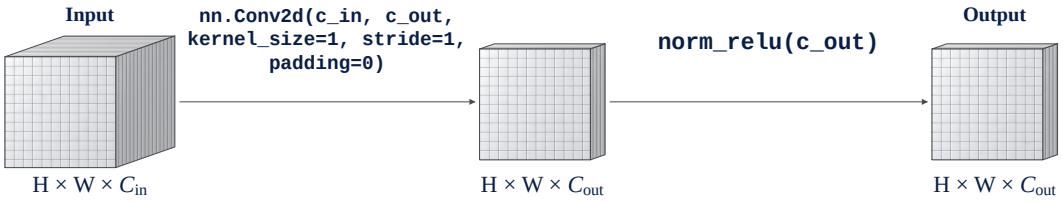
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustrations

2D Pointwise Unit

`unit_2d_pointwise(c_in, c_out)`




```python
def unit_2d_pointwise(c_in, c_out):
    conv = nn.Conv2d(c_in, c_out, kernel_size=1, stride=1, padding=0)
    return nn.Sequential(conv, *norm_relu(c_out))
```

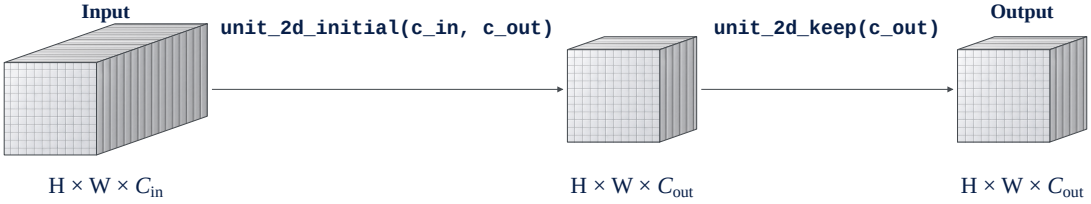
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Decoder Block

`decoder_block(c_in, c_out)`




```
def decoder_block(c_in, c_out):
    return nn.Sequential(unit_2d_initial(c_in, c_out),
                         unit_2d_keep(c_out))
```

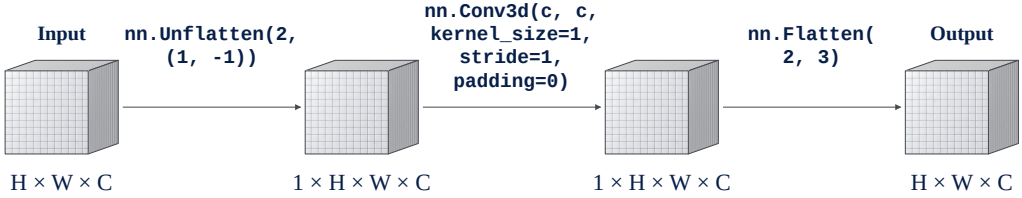
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

3D Convolution

unit_3d(c)




```
def unit_3d(c):
    unflatten = nn.Unflatten(2, (1, -1))
    conv = nn.Conv3d(c, c, kernel_size=1, stride=1, padding=0)
    flatten = nn.Flatten(2, 3)
    return nn.Sequential(unflatten, conv, flatten)
```

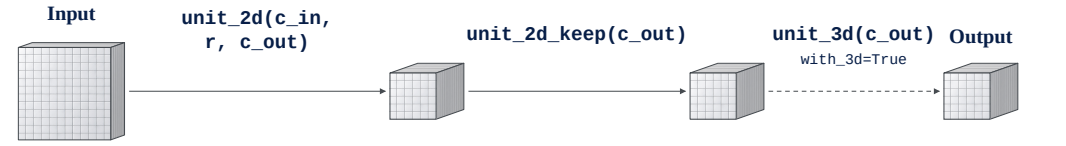
Tensor shape: $D \times H \times W \times C$

D = depth (1), H = height, W = width, C = channels

Source: AI generated illustration

Encoder Stage

`encoder_stage(c_in, c_out, r=2, with_3d=False)`




```python
def encoder_stage(c_in, c_out, r=2, with_3d=False):
    layers = [unit_2d(c_in, r, c_out), unit_2d_keep(c_out)]
    if with_3d:
        layers.append(unit_3d(c_out))
    return nn.Sequential(*layers)
```

Tensor shape: $H \times W \times C$

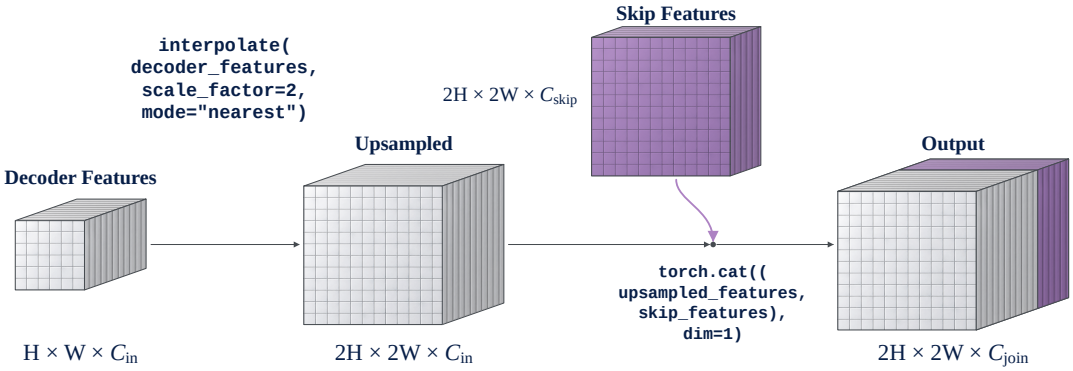
H = height, W = width, C = channels

Source: AI generated illustration

Upsample and Concat

```
upsample_concat(decoder_features, skip_features)
```

$$C_{\text{join}} = C_{\text{in}} + C_{\text{skip}}.$$



Tensor shape: $H \times W \times C$

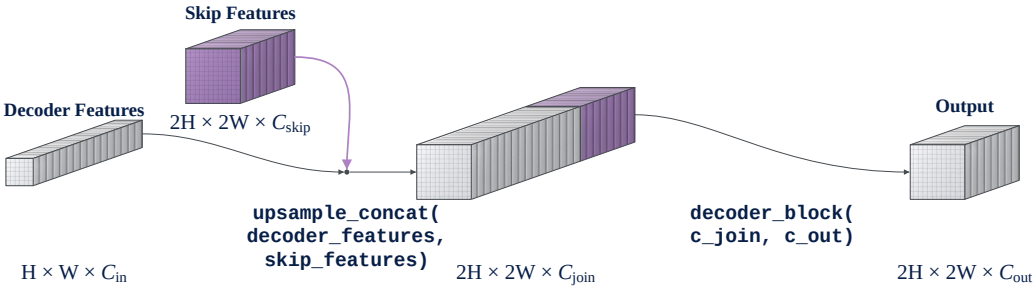
H = height, W = width, C = channels

Source: AI generated illustration

Decoder Stage

```
decoder_stage(c_in, c_skip, c_out)
```

$$C_{\text{join}} = C_{\text{in}} + C_{\text{skip}}.$$



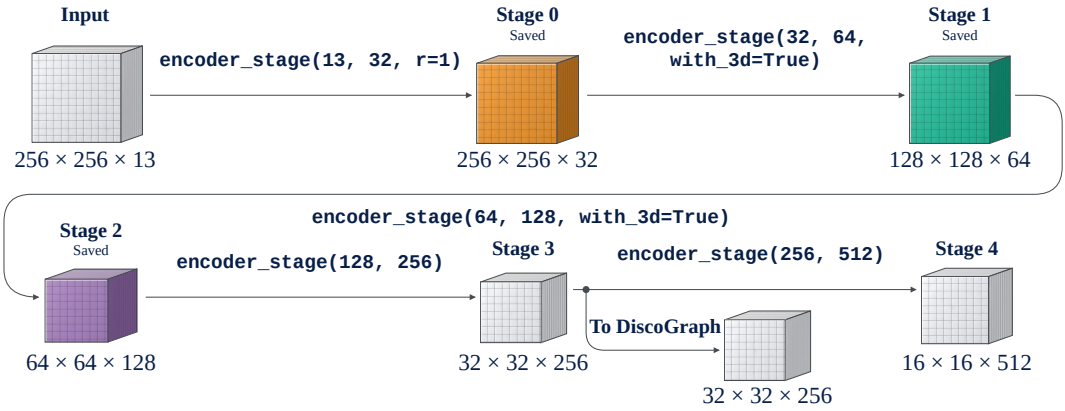
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Encoder

Encoder Pipeline



Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Encoder Code

Simplified PyTorch-style architecture.

Encoder

```
encoder = Sequential(
    Conv2d(13, 32, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(32),
    ReLU(),
    Conv2d(32, 32, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(32),
    ReLU(),
    Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(64),
    ReLU(),
    Conv2d(64, 64, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(64),
    ReLU(),
    Unflatten(2, (1, -1)),
    Conv3d(64, 64, kernel_size=1, stride=1, padding=0),
    Flatten(2, 3),
    Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(128),
    ReLU(),
```

Encoder (Continued)

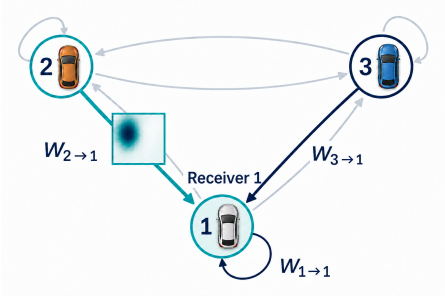
```
    Conv2d(128, 128, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(128),
    ReLU(),
    Unflatten(2, (1, -1)),
    Conv3d(128, 128, kernel_size=1, stride=1, padding=0),
    Flatten(2, 3),
    Conv2d(128, 256, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(256),
    ReLU(),
    Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(256),
    ReLU(),
    Conv2d(256, 512, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(512),
    ReLU(),
    Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(512),
    ReLU(),
)
```

Source: Adapted from the appendix of Li et al.

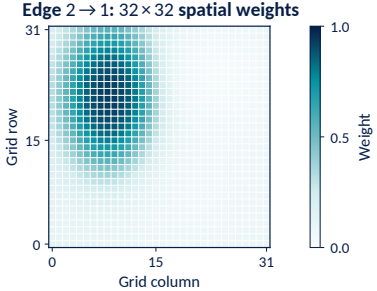
DiscoGraph

DiscoGraph

For $u \to v$, cell (r, c) is a small region in receiver v's top-down feature map, in v's coordinates. Sender u's feature map is aligned to v's feature map so both describe the same physical locations.



At each cell, v combines all agents' information with normalized weights: $\sum_{u=1}^{n} w_{u\to v}(r, c) = 1$.

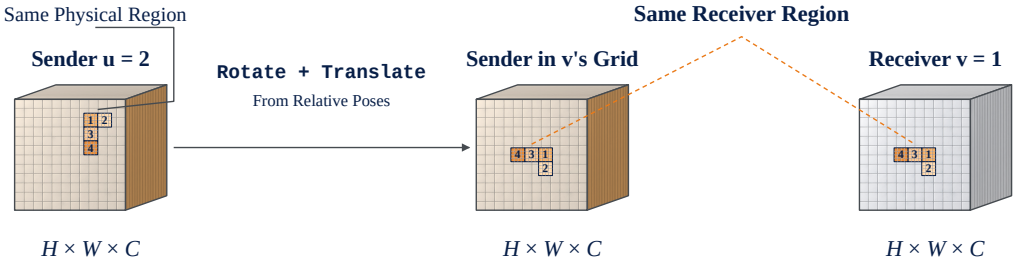


Higher weights give an agent more influence.

Source: AI generated illustration

DiscoGraph: Feature Alignment

Rotate and translate sender u's feature map into receiver v's coordinates.



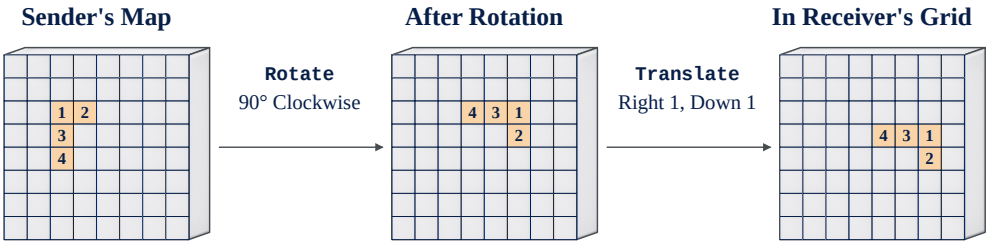
All channels use the same transform.

Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Rotate and Translate



Example: 8×8 map, one feature channel.

Source: AI generated illustration

DiscoGraph: Alignment in PyTorch

1. `align_features` ⟶

```
import math
from torch.nn import functional as F

def align_features(
    sender_features, angle_deg, dx, dy
):
    _, C, H, W = sender_features.shape
    a = math.radians(angle_deg)
    cos, sin = math.cos(a), math.sin(a)
    transform = sender_features.new_tensor([
        [cos, -sin, 2 * dx / W],
        [sin,  cos, 2 * dy / H],
        [0,    0,    1],
    ])
```

2. `align_features` **(Continued)**

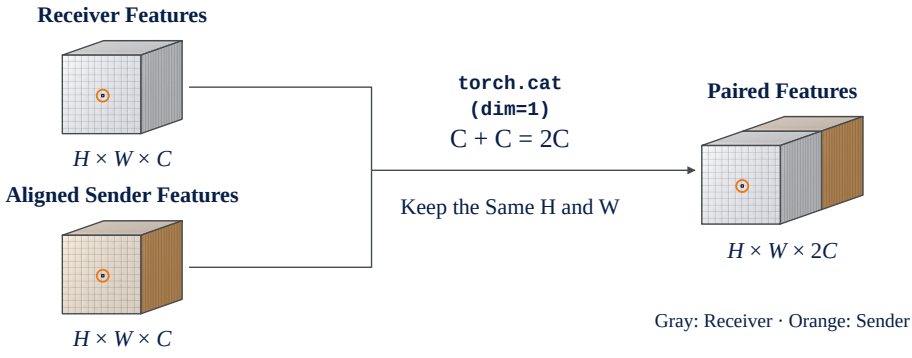
```
    inverse = transform.inverse()
    theta = inverse[:2].unsqueeze(0)
    grid = F.affine_grid(
        theta, (1, C, H, W),
        align_corners=False)
    return F.grid_sample(
        sender_features, grid,
        align_corners=False)
```

Same Example: $90°$**, Right 1, Down 1**

```
aligned_features = align_features(
    sender_features, angle_deg=90, dx=1, dy=1
)
```

Source: PyTorch affine_grid and grid_sample documentation

DiscoGraph: Pair the Features

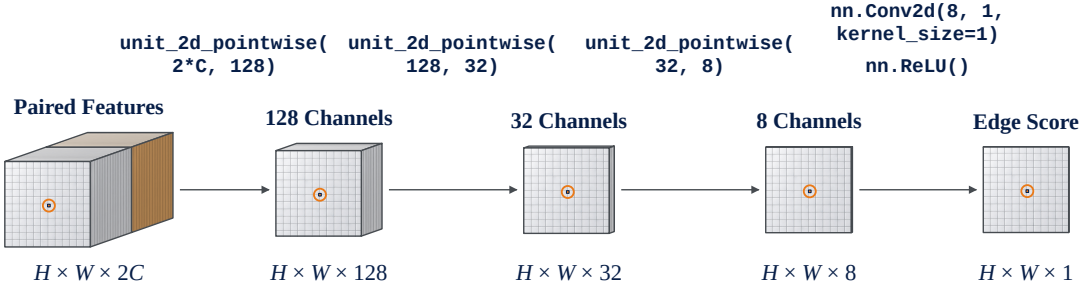


Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Edge Scores



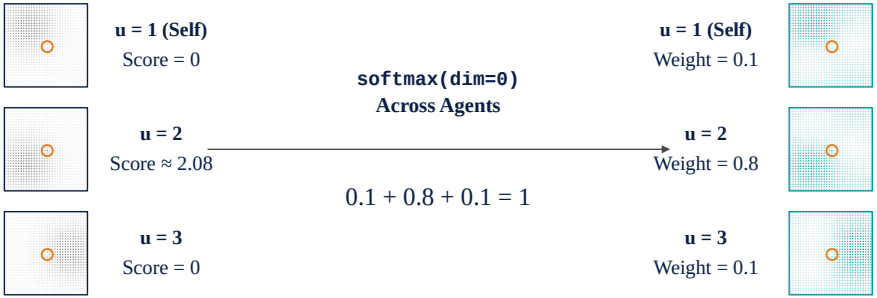
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Spatial Weights

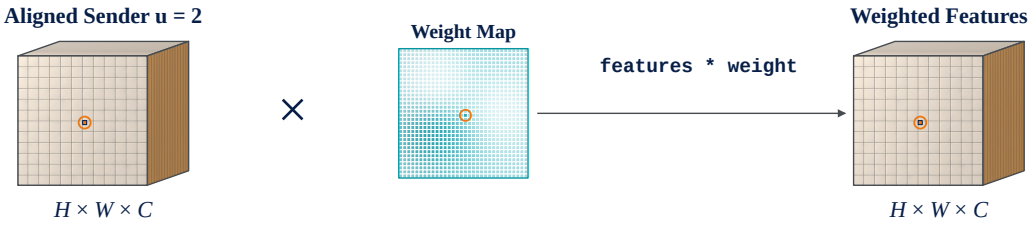
$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$



Source: AI generated illustration

DiscoGraph: Weighted Features

Multiply the aligned sender's entire C-channel feature vector by its weight at that cell.



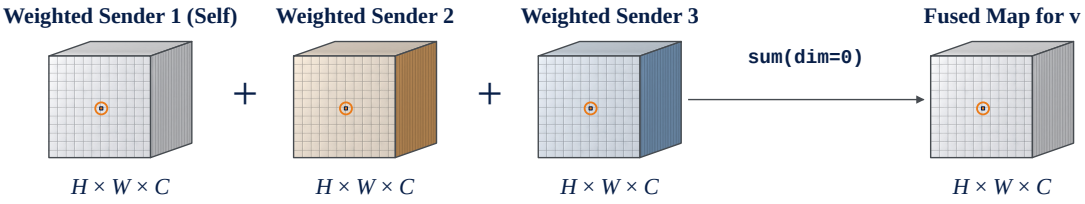
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Fused Features

Add the weighted contributions from all senders, including the receiver itself.



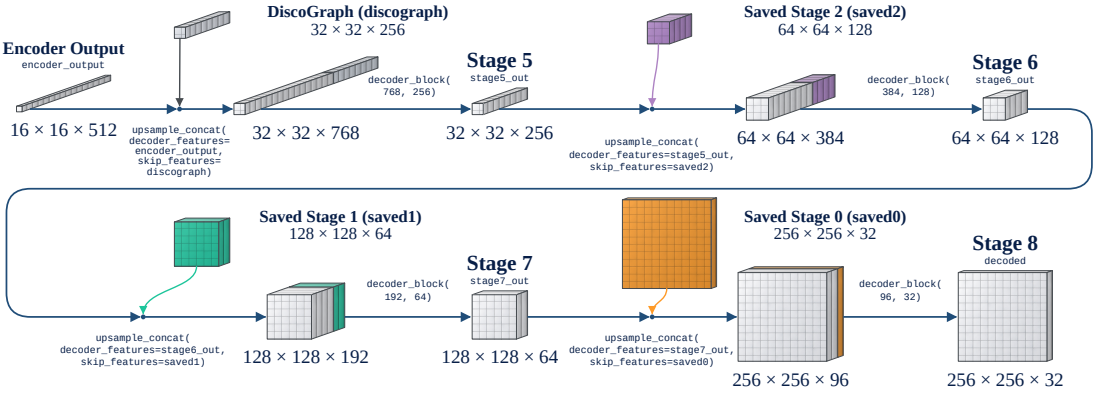
Tensor shape: $H \times W \times C$

Source: AI generated illustration

H = height, W = width, C = channels

Decoder

Decoder Pipeline



Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Experiments

Intersection over Union (IoU)

Compare a predicted box with its ground-truth (reference) box, viewed from above.

$$\text{IoU} = \frac{\text{shared area (intersection)}}{\text{total area covered (union)}} = \frac{6}{8+8-6} = \frac{6}{10} = 0.60 = 60\%$$

Each square is one area unit. IoU ranges from 0 (no overlap) to 1 (identical boxes).

Source: AI generated illustrations

IoU: Matching Thresholds

In the literature, IoU is usually associated with a matching threshold.

Example A: IoU = 0.33 **Example B: IoU = 0.60** **Example C: IoU = 0.80**

Minimum IoU	A: 0.33	B: 0.60	C: 0.80
0.5 (50%)	Fail	Match	Match
0.7 (70%)	Fail	Match	Match

Source: AI generated illustrations

DiscoNet Confidence Scores

1. Predict Raw Outputs **2. Convert With Softmax** **3. Keep and Rank Boxes**

$$\text{score} = \frac{e^{z_{\text{car}}}}{e^{z_{\text{background}}} + e^{z_{\text{car}}}}$$

Illustrative Example: Raw Outputs to Scores

Box	Background Logit	Car Logit	Car Score	Score > 0.7?
A	1.0	3.0	0.881 (88.1%)	Keep
B	2.0	1.0	0.269 (26.9%)	Discard
C	1.0	4.0	0.953 (95.3%)	Keep

Box A: Apply the Formula

$$\frac{e^3}{e^1 + e^3} \approx 0.881$$

88.1% car confidence

Source: AI generated illustration

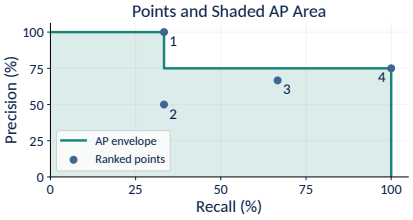
Precision, Recall, and AP

Rank by confidence (car score). TP = true positive (correct match), FP = false positive (unmatched/duplicate).

$$\text{Precision} = \frac{\text{TP}}{\text{TP+FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{reference cars}} \qquad \text{IoU} \geq 0.5$$

Illustrative Example: 3 Reference Cars

Top k	Score	Result	TP / FP So Far	Precision	Recall
1	0.99	TP	1 / 0	100.0%	33.3%
2	0.96	FP	1 / 1	50.0%	33.3%
3	0.92	TP	2 / 1	66.7%	66.7%
4	0.83	TP	3 / 1	75.0%	100.0%



$$\text{AP} = \frac{1}{3} \times 100\% + \frac{1}{3} \times 75\% + \frac{1}{3} \times 75\% = \mathbf{83.3\%}$$

Source: AI generated illustration

V2X-Sim 2.0: Dataset Analysis

100 scenes, 10,000 synchronized frames and 37,200 vehicle scans; roadside unit excluded.

Official Partitions

Partition	Scenes	Frames	Scans	Points
Train	80	8,000	29,200	1.0 B
Validation	10	1,000	4,000	106.8 M
Test	10	1,000	4,000	100.8 M
Total	**100**	**10,000**	**37,200**	**1.2 B**

One scan = one vehicle at one frame.

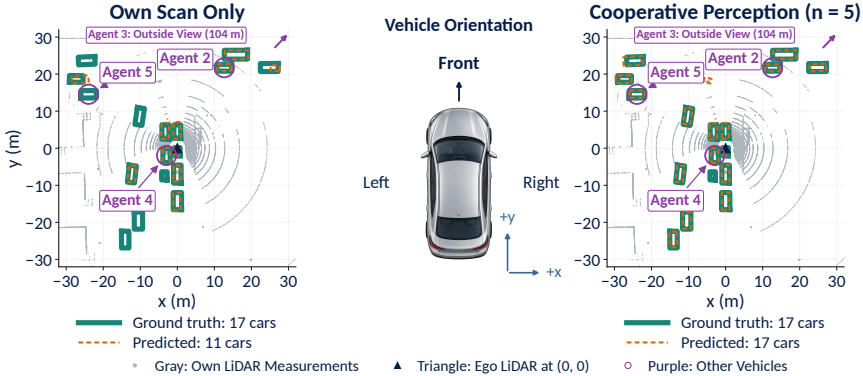
Grouped by Vehicle Inputs

Partition	Inputs	Scenes	Frames	Scans	Points
Train	2	4	400	800	28.9 M
	3	37	3,700	11,100	375.4 M
	4	22	2,200	8,800	294.8 M
	5	17	1,700	8,500	333.9 M
Validation	3	3	300	900	22.0 M
	4	4	400	1,600	48.4 M
	5	3	300	1,500	36.4 M
Test	3	3	300	900	22.7 M
	4	4	400	1,600	41.0 M
	5	3	300	1,500	37.1 M

Source: AI generated illustrations

V2X-Sim 2.0: Car Detections

Example 1: scene_5, raw frame 000066; vehicle 1, five participating vehicles.



Front is $+y$, right is $+x$; $(0, 0)$ is the LiDAR sensor. References can include cars hidden from its scan.

Source: AI generated illustration

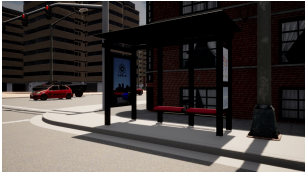
V2X-Sim 2.0: Camera Views

Six synchronized views provide context; the detector uses LiDAR. Camera and LiDAR positions differ.

Front Left

Front

Front Right



Rear Left

Rear

Rear Right

Source: V2X-Sim 2.0

Experiment Setup

Configuration of the completed 1,000-frame benchmark.

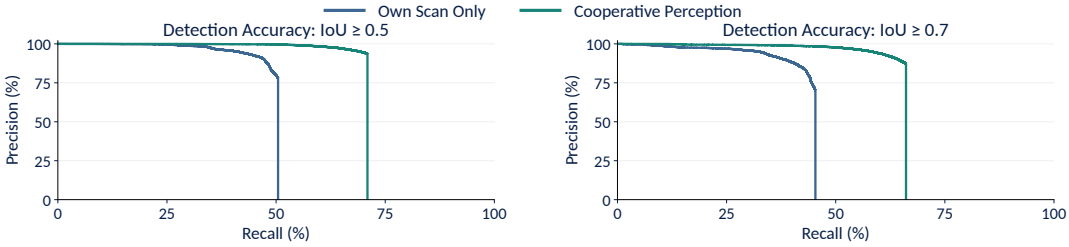
Hardware	
GPU	NVIDIA GeForce RTX 5060 Ti
GPU Memory	16 GB
CPU	AMD Ryzen 7 5700X
CPU Cores	8 cores / 16 threads
System RAM	16 GB

Software	
OS	Omarchy (Linux)
Python	3.14.8
PyTorch	2.9.1+cu128
CUDA Runtime	12.8

Source: AI generated illustrations

V2X-Sim 2.0: Reproduced Results

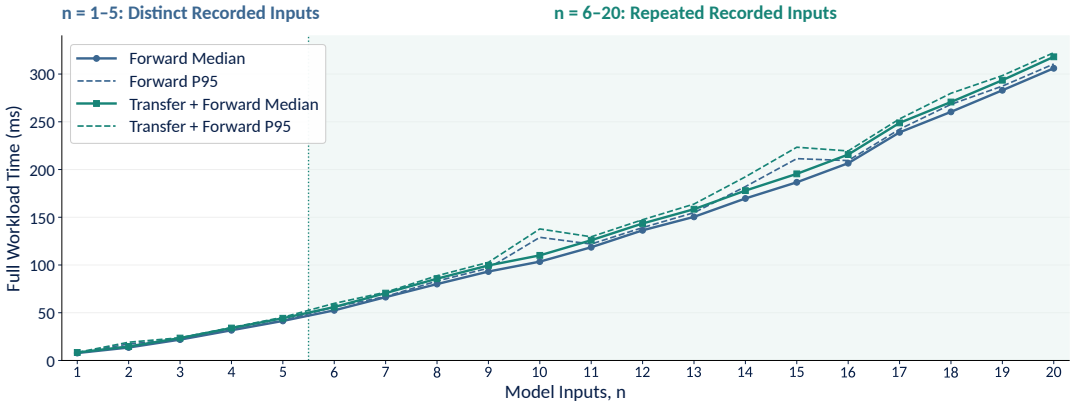
Condition	AP@0.5 (%)	AP@0.7 (%)
Own scan only	49.10	43.05
Cooperative perception	**70.41**	**64.82**



Own scan only switches information sharing off; both conditions use the same weights.
Sharing improves AP by 21.31 and 21.77 percentage points, respectively.

Source: AI generated illustrations

V2X-Sim 2.0: Runtime Scaling



Source: AI generated illustration

Next Steps

Next Steps

Immediate next actions

Identify further useful extensions and explore more modern, complex approaches.

Expected deliverables

A shortlist of extension ideas and a proposed experiment.

Priority order

Finish writing the final EQM version and dissertation; resolve the observations on the first project deliverable by Friday; find suitable extensions; then study a new approach.

Timeline

First, finish the EQM and dissertation writing; by Friday, resolve the deliverable observations; then shortlist extensions and study a new approach.

Next Steps

Validation plan

Review the final EQM and dissertation drafts, confirm that each deliverable observation is addressed, and define an experiment for the selected extension.

Risks or dependencies

The complexity of the new approach.

Questions for discussion

How large can these models be, and how feasible is it for us to train them from scratch?

Thank you

Questions?

Appendix: Conv2D vs Conv3D

Initial Tensor

Assumptions

Batch size

$N = 1$

Channels $C_{\text{in}} \rightarrow C_{\text{out}}$

$3 \rightarrow 1$

Video $D \times H \times W$

$3 \times 3 \times 4$

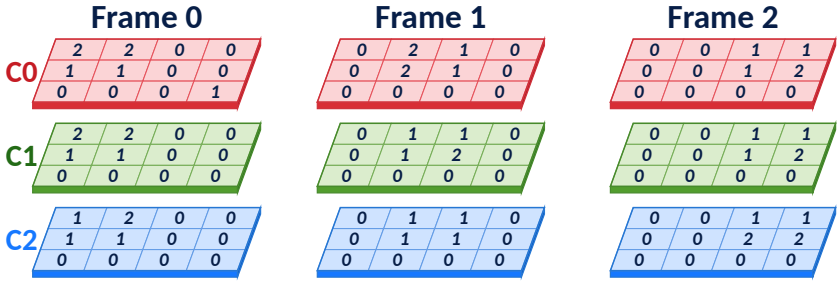
PyTorch defaults

Groups	1
Stride	1
Padding	0
Dilation	1

Bias = **0**

Set for this example

Consider the following video tensor

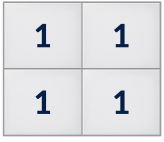


In this example, **Conv2D** uses the first frame; **Conv3D** uses the whole tensor

Kernels

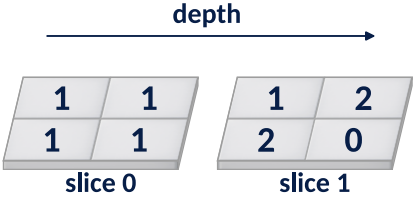
Use the shown weights in each of the three input channels. Bias $= 0$

Conv2D



Kernel size: $(2, 2)$

Conv3D

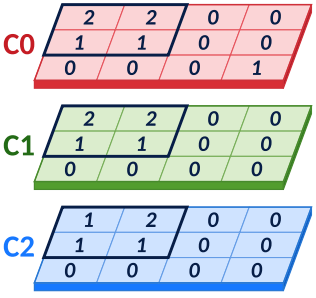


Kernel size: $(2, 2, 2)$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 0)$: apply the 2×2 kernel to each channel of Frame 0

Frame 0



Kernel

1	1
1	1

Output

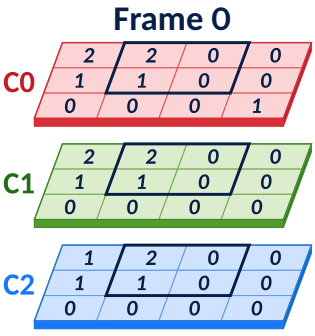
17	9	0
6	3	1

C0: $2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 6$ **C1:** $2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 6$

C2: $1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 5$ **Output:** $6 + 6 + 5 = \boxed{17}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 1)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

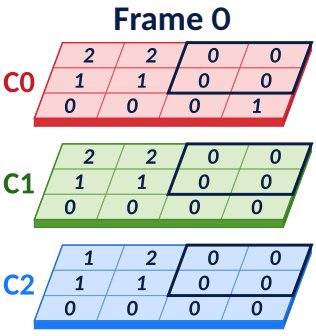
17	**9**	0
6	3	1

C0: $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$ **C1:** $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$

C2: $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$ **Output:** $3 + 3 + 3 = \boxed{9}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 2)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

17	9	0
6	3	1

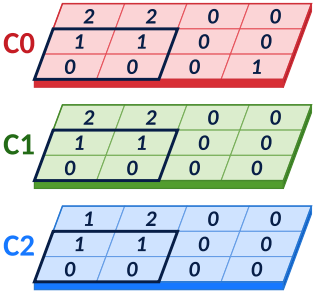
C0: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **C1:** $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$

C2: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **Output:** $0 + 0 + 0 = \boxed{0}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 0)$: apply the 2×2 kernel to each channel of Frame 0

Frame 0



Kernel

1	1
1	1

Output

17	9	0
6	3	1

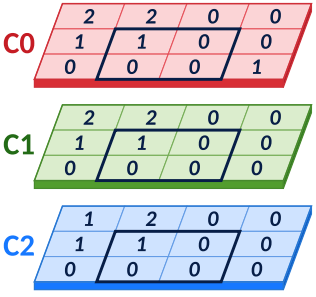
C0: $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$ **C1:** $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$

C2: $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$ **Output:** $2 + 2 + 2 = \boxed{6}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 1)$: apply the 2×2 kernel to each channel of Frame 0

Frame 0



Kernel

1	1
1	1

Output

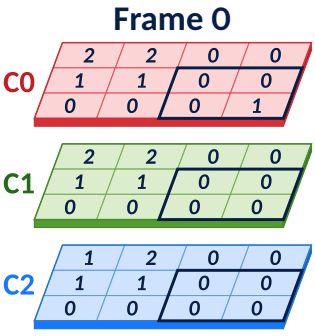
17	9	0
6	**3**	1

C0: $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$ **C1:** $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$

C2: $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$ **Output:** $1 + 1 + 1 = \boxed{3}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 2)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

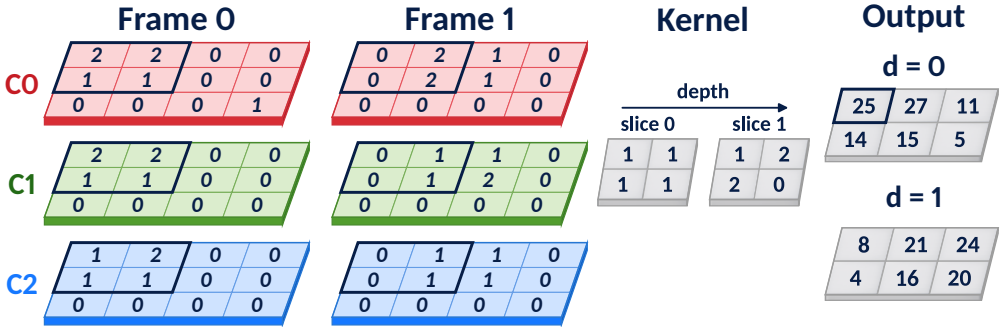
17	9	0
6	3	1

C0: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 1$ **C1:** $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$

C2: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **Output:** $1 + 0 + 0 = \boxed{1}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 0)$: use the same two-slice kernel in each channel

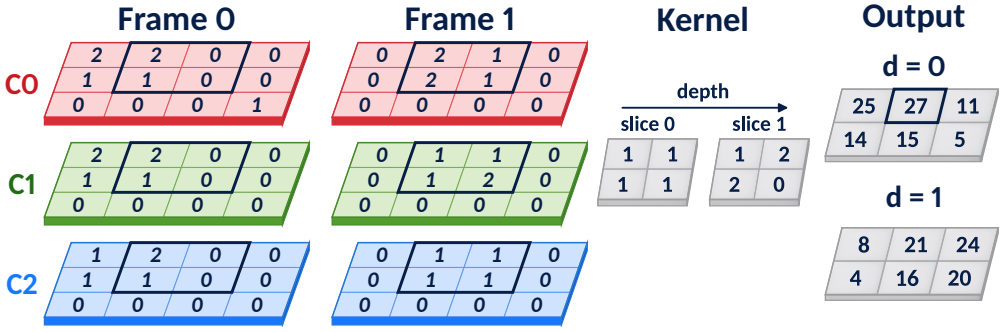


C0: $(2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1) + (0 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 2 \cdot 0) = 6 + 4 = 10$

C1: $6 + 2 = 8$; **C2:** $5 + 2 = 7$ **Output:** $10 + 8 + 7 = \boxed{25}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 1)$: use the same two-slice kernel in each channel

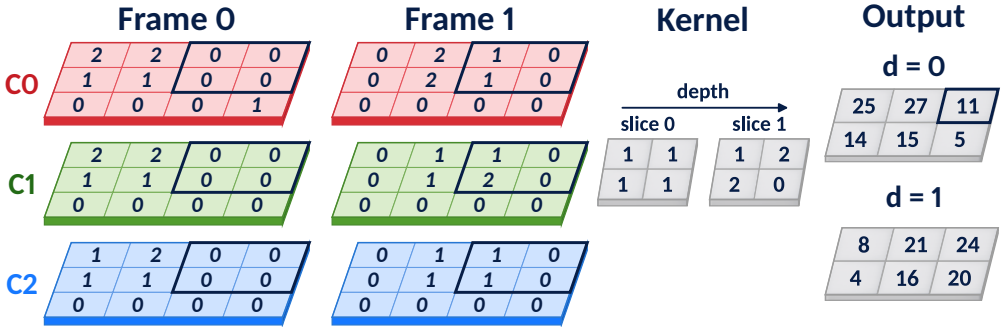


C0: $(2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1) + (2 \cdot 1 + 1 \cdot 2 + 2 \cdot 2 + 1 \cdot 0) = 3 + 8 = 11$

C1: $3 + 5 = 8$; **C2:** $3 + 5 = 8$ **Output:** $11 + 8 + 8 = \boxed{27}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 2)$: use the same two-slice kernel in each channel

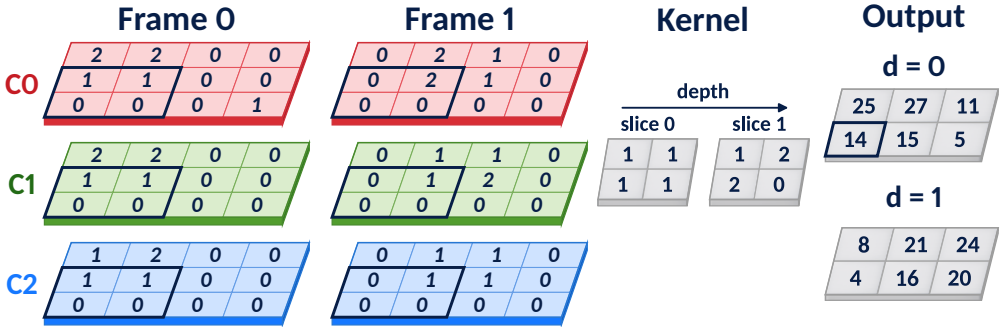


C0: $(0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 0 \cdot 2 + 1 \cdot 2 + 0 \cdot 0) = 0 + 3 = 3$

C1: $0 + 5 = 5$; **C2:** $0 + 3 = 3$ **Output:** $3 + 5 + 3 = \boxed{11}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 0)$: use the same two-slice kernel in each channel

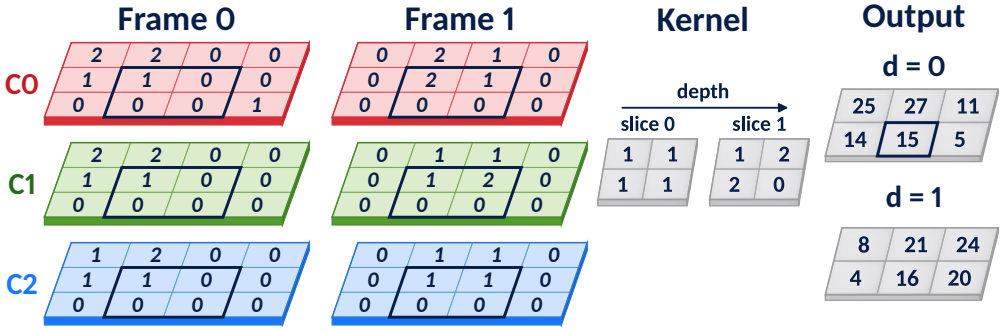


C0: $(1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 2 + 4 = 6$

C1: $2 + 2 = 4$; **C2:** $2 + 2 = 4$ **Output:** $6 + 4 + 4 = \boxed{14}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 1)$: use the same two-slice kernel in each channel

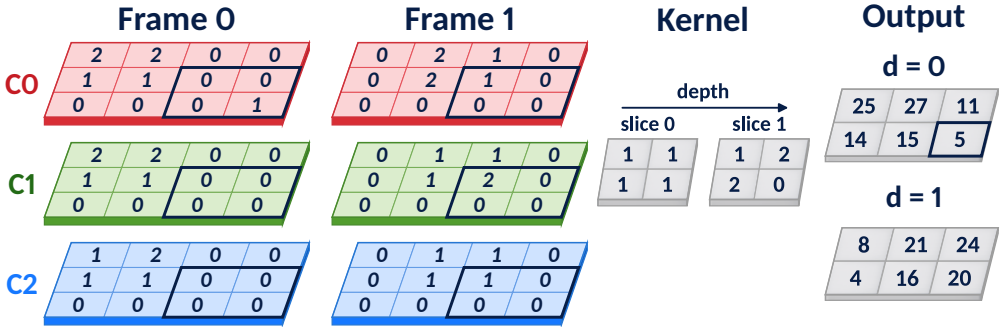


C0: $(1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (2 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 4 = 5$

C1: $1 + 5 = 6$; **C2:** $1 + 3 = 4$ **Output:** $5 + 6 + 4 = \boxed{15}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 2)$: use the same two-slice kernel in each channel

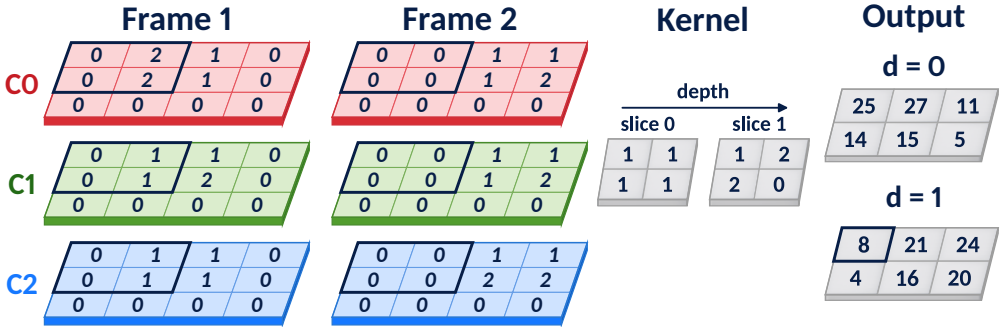


C0: $(0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1) + (1 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 1 = 2$

C1: $0 + 2 = 2$; **C2:** $0 + 1 = 1$ **Output:** $2 + 2 + 1 = \boxed{5}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 0)$: use the same two-slice kernel in each channel

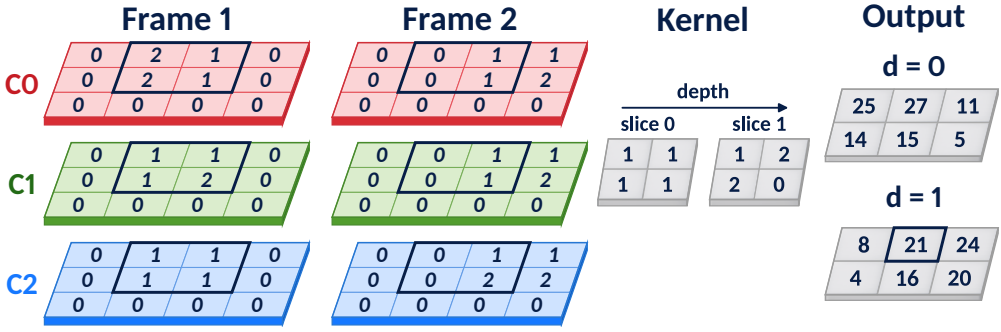


C0: $(0 \cdot 1 + 2 \cdot 1 + 0 \cdot 1 + 2 \cdot 1) + (0 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 4 + 0 = 4$

C1: $2 + 0 = 2$; **C2:** $2 + 0 = 2$ **Output:** $4 + 2 + 2 = \boxed{8}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 1)$: use the same two-slice kernel in each channel

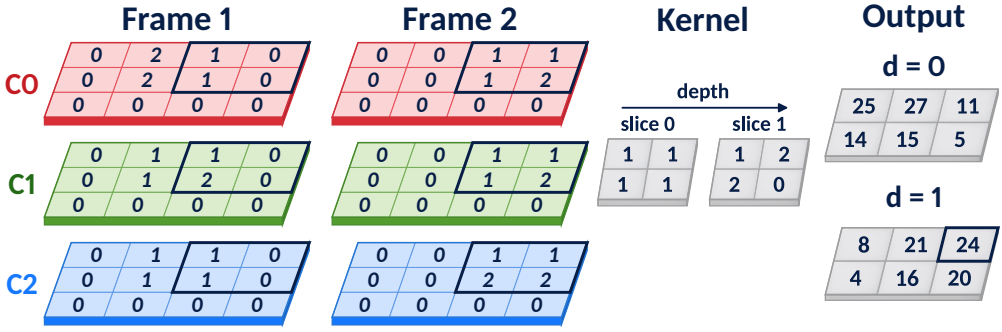


C0: $(2 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1) + (0 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 1 \cdot 0) = 6 + 2 = 8$

C1: $5 + 2 = 7$; **C2:** $4 + 2 = 6$ **Output:** $8 + 7 + 6 = \boxed{21}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 2)$: use the same two-slice kernel in each channel

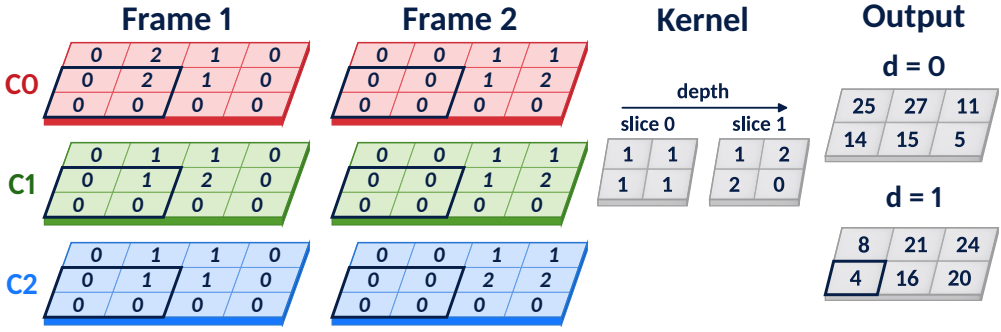


C0: $(1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 1 \cdot 2 + 1 \cdot 2 + 2 \cdot 0) = 2 + 5 = 7$

C1: $3 + 5 = 8$; **C2:** $2 + 7 = 9$ **Output:** $7 + 8 + 9 = \boxed{24}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 0)$: use the same two-slice kernel in each channel

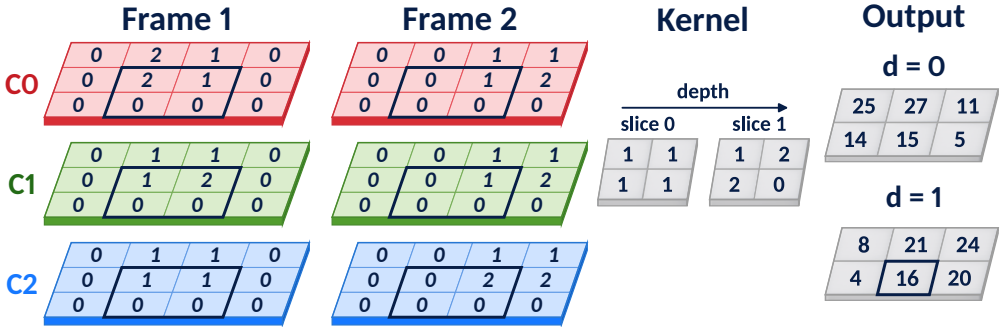


C0: $(0 \cdot 1 + 2 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 2 + 0 = 2$

C1: $1 + 0 = 1$; **C2:** $1 + 0 = 1$ **Output:** $2 + 1 + 1 = \boxed{4}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 1)$: use the same two-slice kernel in each channel

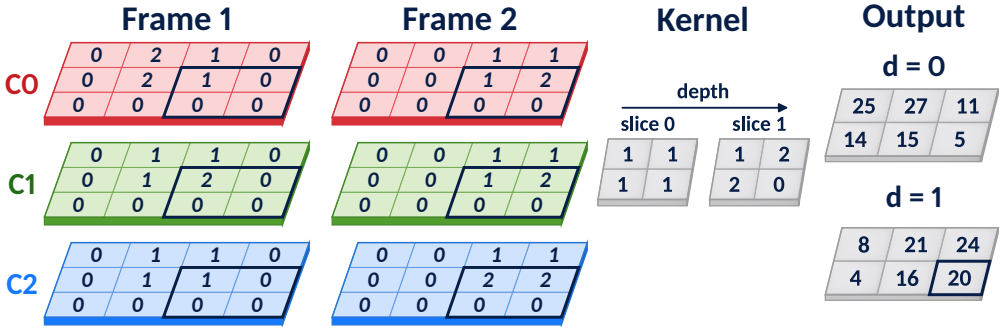


C0: $(2 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 3 + 2 = 5$

C1: $3 + 2 = 5$; **C2:** $2 + 4 = 6$ **Output:** $5 + 5 + 6 = \boxed{16}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 2)$: use the same two-slice kernel in each channel



C0: $(1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 5 = 6$

C1: $2 + 5 = 7$; **C2:** $1 + 6 = 7$ **Output:** $6 + 7 + 7 = \boxed{20}$